

03. Semi-Supervised Learning: Self-Training Algorithm

Algorithm Covered:

- **Self-Training Classifier** (Iterative Pseudo-Labeling with Base Estimators)

Core Focus:

- Exploiting large volumes of inexpensive unlabeled data alongside limited labeled data
- Iterative confidence thresholding (τ) and pseudo-labeling dynamics
- Comparative evaluation: **Supervised Baseline (15% labeled)** vs. **Semi-Supervised Self-Training** vs. **Fully Supervised Ceiling (100% labeled)**
- Base Estimators: Support Vector Classifier (RBF Kernel) & Random Forest Classifier

```
In [1]: # ---- Suppress a known sklearn deprecation notice for SVC(probability=True) ----
import warnings
# ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Dataset + train/test utilities ----
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
# ---- Semi-supervised self-training wrapper ----
from sklearn.semi_supervised import SelfTrainingClassifier
# ---- Base estimators compared inside self-training ----
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
# ---- Evaluation metrics ----
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
roc_auc_score, confusion_matrix, classification_report

# Aesthetics setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available
else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

# The installed scikit-learn still supports SVC(probability=True) but warns; silence that
specific notice
warnings.filterwarnings("ignore", message="The `probability` parameter was deprecated",
category=FutureWarning)
np.random.seed(42)
print("Semi-supervised libraries imported successfully!")
```

Semi-supervised libraries imported successfully!

1. Dataset Selection and Semi-Supervised Masking

We use the **Breast Cancer Wisconsin** dataset (30 continuous diagnostic features, 569 samples, binary classification: Malignant vs. Benign).

Simulating Semi-Supervised Conditions:

- 30% held out as a strict, clean **Test Set** (171 samples).
- 70% used for training (398 samples).
- In the training set, **85% of samples are masked as unlabeled** (assigned label `-1` in Scikit-Learn convention).
- Only **15% remain labeled** (~60 samples) as ground truth seeds.

```
In [2]: # ---- Load the Breast Cancer Wisconsin dataset ----
data = load_breast_cancer()
X_full = data.data           # 30 continuous diagnostic features
y_full = data.target         # 0 = malignant, 1 = benign
feature_names = data.feature_names
target_names = data.target_names

# ---- Train / Test split (stratified to preserve class balance) ----
X_train_full, X_test, y_train_full, y_test = train_test_split(
    X_full, y_full, test_size=0.30, random_state=42, stratify=y_full
)

# ---- Standardize features (fit on train only to avoid test leakage) ----
scaler = StandardScaler()
X_train_full_scaled = scaler.fit_transform(X_train_full)
X_test_scaled = scaler.transform(X_test)

# ---- Simulate the semi-supervised setting: hide 85% of the training Labels ----
unlabeled_ratio = 0.85
rng = np.random.RandomState(42)
random_unlabeled_points = rng.rand(len(y_train_full)) < unlabeled_ratio # True -> mask

y_train_semi = np.copy(y_train_full)
y_train_semi[random_unlabeled_points] = -1 # -1 is the scikit-learn convention for
"unlabeled"

n_labeled = np.sum(y_train_semi != -1)
n_unlabeled = np.sum(y_train_semi == -1)

print(f"Total Training Samples:    {len(y_train_full)}")
print(f" - Labeled Samples (15%): {n_labeled}")
print(f" - Unlabeled Samples (85%): {n_unlabeled}")
print(f"Holdout Test Samples:         {len(y_test)}")
```

```
Total Training Samples:    398
- Labeled Samples (15%): 61
- Unlabeled Samples (85%): 337
Holdout Test Samples:      171
```

2. Mathematical Formulation of Self-Training

Given labeled set $L = \{(x_i, y_i)\}_{i=1}^l$ and unlabeled pool $U = \{x_j\}_{j=1}^u$:

1. Train probabilistic classifier f on L .

2. For each sample $x \in U$, compute class probability distribution:

$$\hat{P}(y | x) = f(x)$$

3. Identify high-confidence predictions exceeding threshold τ :

$$U^* = \left\{ (x, \hat{y}) \mid x \in U, \hat{y} = \arg \max_y \hat{P}(y | x), \max_y \hat{P}(y | x) \geq \tau \right\}$$

4. Augment labeled set: $L \leftarrow L \cup U^*$ and remove from unlabeled pool: $U \leftarrow U \setminus U^*$.

5. Repeat steps 1–4 until convergence (no points exceed τ or maximum iterations reached).

Pseudocode (Self-Training Classifier)

```
L = labeled set          # 15% of training data
U = unlabeled pool       # 85% of training data (labels hidden)
repeat until no new confident labels or max_iter reached:
    f = train(base_estimator, L)          # e.g. SVC or Random Forest
    P = f.predict_proba(U)                # class probabilities for each
    unlabeled point
    S = { (x, argmax P(x)) : max P(x) >= threshold } # confident pseudo-labels
    L = L union S                          # augment labeled set
    U = U minus S                          # remove newly labeled points
return f
```

Key hyperparameter: the confidence threshold τ controls the purity/quantity trade-off of pseudo-labels.

3. Experiment 1: Self-Training with Support Vector Classifier (RBF Kernel)

We compare:

1. **Supervised Baseline:** SVC trained strictly on the 60 labeled samples.
2. **Semi-Supervised Self-Training:** SVC trained on 60 labeled + 338 unlabeled samples via iterative pseudo-labeling.
3. **Fully Supervised Ceiling:** SVC trained on all 398 ground truth labeled samples.

```
In [3]: # ---- Experiment 1: Self-Training with an RBF-kernel SVC ----

# 1. Supervised baseline trained ONLY on the ~60 labeled samples
mask_labeled = (y_train_semi != -1)          # boolean mask of the labeled subset
X_labeled = X_train_full_scaled[mask_labeled]
y_labeled = y_train_semi[mask_labeled]

base_svc = SVC(kernel='rbf', probability=True, C=1.0, random_state=42)
base_svc.fit(X_labeled, y_labeled)
y_pred_baseline_svc = base_svc.predict(X_test_scaled)
acc_baseline_svc = accuracy_score(y_test, y_pred_baseline_svc)
f1_baseline_svc = f1_score(y_test, y_pred_baseline_svc)

# 2. Semi-supervised self-training: base SVC + iterative pseudo-labeling above threshold 0.80
self_training_svc = SelfTrainingClassifier(
    estimator=SVC(kernel='rbf', probability=True, C=1.0, random_state=42),
    threshold=0.80,          # only accept pseudo-labels with confidence >= 0.80
    criterion='threshold',   # use the confidence-threshold selection rule
```

```

max_iter=15,
verbose=True                # print how many labels are added per iteration
)
self_training_svc.fit(X_train_full_scaled, y_train_semi)
y_pred_semi_svc = self_training_svc.predict(X_test_scaled)
acc_semi_svc = accuracy_score(y_test, y_pred_semi_svc)
f1_semi_svc = f1_score(y_test, y_pred_semi_svc)

# 3. Fully supervised "ceiling": train on 100% of the ground-truth labels
full_svc = SVC(kernel='rbf', probability=True, C=1.0, random_state=42)
full_svc.fit(X_train_full_scaled, y_train_full)
y_pred_full_svc = full_svc.predict(X_test_scaled)
acc_full_svc = accuracy_score(y_test, y_pred_full_svc)
f1_full_svc = f1_score(y_test, y_pred_full_svc)

# ---- Compare the three settings ----
print(f"\n--- SVC Evaluation Results ---")
print(f"Supervised Baseline (15% labeled): Accuracy = {acc_baseline_svc:.4f}, F1 = {f1_baseline_svc:.4f}")
print(f"Self-Training (Semi-supervised): Accuracy = {acc_semi_svc:.4f}, F1 = {f1_semi_svc:.4f}")
print(f"Fully Supervised Ceiling (100%): Accuracy = {acc_full_svc:.4f}, F1 = {f1_full_svc:.4f}")

```

End of iteration 1, added 292 new labels.

End of iteration 2, added 22 new labels.

End of iteration 3, added 5 new labels.

End of iteration 4, added 2 new labels.

--- SVC Evaluation Results ---

Supervised Baseline (15% labeled): Accuracy = 0.9298, F1 = 0.9469

Self-Training (Semi-supervised): Accuracy = 0.9415, F1 = 0.9554

Fully Supervised Ceiling (100%): Accuracy = 0.9766, F1 = 0.9813

```

In [4]: # ---- Inspect how self-training converged ----
# n_iter_                : number of self-training iterations performed
# termination_condition_ : why training stopped (e.g. 'no_change' = no new confident labels)
# transduction_          : final labels for every training sample (-1 = still unlabeled)
n_iter = self_training_svc.n_iter_
termination_condition = self_training_svc.termination_condition_
labeled_final_mask = (self_training_svc.transduction_ != -1)

print(f"Self-Training Convergence Summary:")
print(f" - Iterations completed: {n_iter}")
print(f" - Termination condition: {termination_condition}")
print(f" - Total samples labeled after self-training: {np.sum(labeled_final_mask)} / {len(y_train_full)}")
print(f" - Pseudo-labels newly added: {np.sum(labeled_final_mask) - n_labeled}")

```

Self-Training Convergence Summary:

- Iterations completed: 5
- Termination condition: no_change
- Total samples labeled after self-training: 382 / 398
- Pseudo-labels newly added: 321

4. Experiment 2: Self-Training with Random Forest Classifier

Let's repeat the benchmark with an ensemble tree estimator (`RandomForestClassifier`) to test generality across different model families.

```
In [5]: # ---- Experiment 2: repeat the benchmark with a Random Forest base estimator ----
```

```
# 1. Supervised baseline on labeled data only
base_rf = RandomForestClassifier(n_estimators=100, random_state=42)
base_rf.fit(X_labeled, y_labeled)
y_pred_baseline_rf = base_rf.predict(X_test_scaled)
acc_baseline_rf = accuracy_score(y_test, y_pred_baseline_rf)
f1_baseline_rf = f1_score(y_test, y_pred_baseline_rf)

# 2. Self-training with Random Forest (slightly higher threshold: 0.85)
self_training_rf = SelfTrainingClassifier(
    estimator=RandomForestClassifier(n_estimators=100, random_state=42),
    threshold=0.85,
    max_iter=15
)
self_training_rf.fit(X_train_full_scaled, y_train_semi)
y_pred_semi_rf = self_training_rf.predict(X_test_scaled)
acc_semi_rf = accuracy_score(y_test, y_pred_semi_rf)
f1_semi_rf = f1_score(y_test, y_pred_semi_rf)

# 3. Fully supervised ceiling on all labels
full_rf = RandomForestClassifier(n_estimators=100, random_state=42)
full_rf.fit(X_train_full_scaled, y_train_full)
y_pred_full_rf = full_rf.predict(X_test_scaled)
acc_full_rf = accuracy_score(y_test, y_pred_full_rf)
f1_full_rf = f1_score(y_test, y_pred_full_rf)

print(f"--- Random Forest Evaluation Results ---")
print(f"Supervised Baseline (15% labeled): Accuracy = {acc_baseline_rf:.4f}, F1 = {f1_baseline_rf:.4f}")
print(f"Self-Training (Semi-supervised): Accuracy = {acc_semi_rf:.4f}, F1 = {f1_semi_rf:.4f}")
print(f"Fully Supervised Ceiling (100%): Accuracy = {acc_full_rf:.4f}, F1 = {f1_full_rf:.4f}")
```

```
--- Random Forest Evaluation Results ---
```

```
Supervised Baseline (15% labeled): Accuracy = 0.9240, F1 = 0.9406
```

```
Self-Training (Semi-supervised): Accuracy = 0.9123, F1 = 0.9321
```

```
Fully Supervised Ceiling (100%): Accuracy = 0.9357, F1 = 0.9488
```

5. Effect of Confidence Threshold (τ) on Model Performance

A crucial hyperparameter in self-training is the pseudo-labeling confidence threshold τ :

- If τ is too low (e.g. 0.60), erroneous pseudo-labels pollute the training pool (**confirmation bias** / error accumulation).
- If τ is too high (e.g. 0.99), too few pseudo-labels are admitted, failing to leverage the unlabeled data.

How the Threshold Sweep Works

1. Choose a grid of thresholds $\tau = 0.65 \dots 0.95$.
2. For each τ : initialize a fresh SelfTrainingClassifier with the same seed.
3. Fit on the same 15%-labeled training set and evaluate on the untouched test set.
4. Record test accuracy, F1, and the final number of labeled samples.
5. Compare against the supervised baseline to see the gain/loss from unlabeled data.

```

In [6]: # ---- Threshold sweep: how does the confidence threshold tau affect performance? ----
thresholds = [0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95]
thresh_accs = []
thresh_f1s = []
thresh_labeled_count = []

for tau in thresholds:
    # Re-run self-training from scratch for every threshold (same seed for fair comparison)
    st_clf = SelfTrainingClassifier(
        estimator=SVC(kernel='rbf', probability=True, C=1.0, random_state=42),
        threshold=tau,
        max_iter=15
    )
    st_clf.fit(X_train_full_scaled, y_train_semi)
    preds = st_clf.predict(X_test_scaled)
    thresh_accs.append(accuracy_score(y_test, preds))
    thresh_f1s.append(f1_score(y_test, preds))
    # Count how many training samples ended up with a Label (true or pseudo)
    thresh_labeled_count.append(np.sum(st_clf.transduction_ != -1))

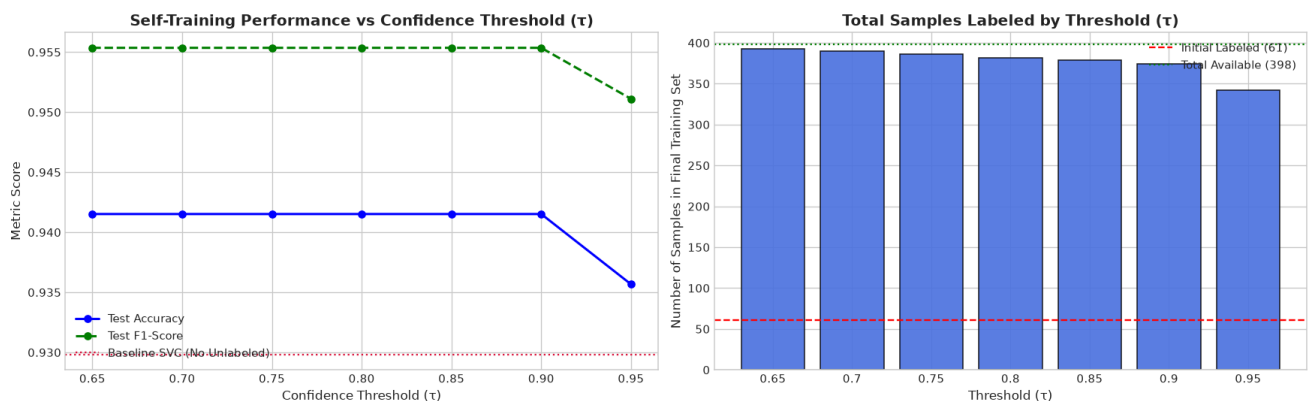
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))

# ---- Panel 1: accuracy and F1 as a function of tau ----
ax1.plot(thresholds, thresh_accs, 'bo-', linewidth=2, label='Test Accuracy')
ax1.plot(thresholds, thresh_f1s, 'go--', linewidth=2, label='Test F1-Score')
ax1.axhline(y=acc_baseline_svc, color='crimson', linestyle=':', label='Baseline SVC (No Unlabeled)')
ax1.set_title("Self-Training Performance vs Confidence Threshold ( $\tau$ )", fontsize=13, fontweight='bold')
ax1.set_xlabel("Confidence Threshold ( $\tau$ )", fontsize=11)
ax1.set_ylabel("Metric Score", fontsize=11)
ax1.legend()

# ---- Panel 2: how many samples receive Labels at each threshold ----
ax2.bar([str(t) for t in thresholds], thresh_labeled_count, color='royalblue', edgecolor='k', alpha=0.85)
ax2.axhline(y=n_labeled, color='red', linestyle='--', label=f'Initial Labeled ({n_labeled})')
ax2.axhline(y=len(y_train_full), color='green', linestyle=':', label=f'Total Available ({len(y_train_full)})')
ax2.set_title("Total Samples Labeled by Threshold ( $\tau$ )", fontsize=13, fontweight='bold')
ax2.set_xlabel("Threshold ( $\tau$ )", fontsize=11)
ax2.set_ylabel("Number of Samples in Final Training Set", fontsize=11)
ax2.legend()

plt.tight_layout()
plt.show()

```



6. Comprehensive Benchmark Summary

Let's synthesize all results into a clean comparative table and visual summary.

Evaluation Process

1. Aggregate the six results (baseline / self-training / ceiling for SVC and Random Forest).
2. Compare Accuracy and F1 in a single highlighted table.
3. Inspect confusion matrices to see which errors self-training removes.

```
In [7]: # ---- Build one summary table with every model/setting ----
summary_data = [
    {"Model": "SVC (Baseline 15% Labeled)", "Accuracy": acc_baseline_svc, "F1-Score":
f1_baseline_svc, "Training Samples": n_labeled},
    {"Model": "SVC (Self-Training Semi-Supervised)", "Accuracy": acc_semi_svc, "F1-Score":
f1_semi_svc, "Training Samples": np.sum(self_training_svc.transduction_ != -1)},
    {"Model": "SVC (Fully Supervised Ceiling 100%)", "Accuracy": acc_full_svc, "F1-Score":
f1_full_svc, "Training Samples": len(y_train_full)},
    {"Model": "Random Forest (Baseline 15% Labeled)", "Accuracy": acc_baseline_rf, "F1-
Score": f1_baseline_rf, "Training Samples": n_labeled},
    {"Model": "Random Forest (Self-Training Semi-Supervised)", "Accuracy": acc_semi_rf, "F1-
Score": f1_semi_rf, "Training Samples": np.sum(self_training_rf.transduction_ != -1)},
    {"Model": "Random Forest (Fully Supervised Ceiling 100%)", "Accuracy": acc_full_rf, "F1-
Score": f1_full_rf, "Training Samples": len(y_train_full)}
]

df_summary = pd.DataFrame(summary_data).set_index("Model")
# Highlight the best Accuracy / F1 in green
display(df_summary.style.highlight_max(subset=['Accuracy', 'F1-Score'], color='lightgreen'))
```

	Accuracy	F1-Score	Training Samples
Model			
SVC (Baseline 15% Labeled)	0.929825	0.946903	61
SVC (Self-Training Semi-Supervised)	0.941520	0.955357	382
SVC (Fully Supervised Ceiling 100%)	0.976608	0.981308	398
Random Forest (Baseline 15% Labeled)	0.923977	0.940639	61
Random Forest (Self-Training Semi-Supervised)	0.912281	0.932127	369
Random Forest (Fully Supervised Ceiling 100%)	0.935673	0.948837	398

```
In [8]: # ---- Confusion-matrix comparison for the SVC setting ----
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Baseline (15% Labeled) confusion matrix
cm_baseline = confusion_matrix(y_test, y_pred_baseline_svc)
sns.heatmap(cm_baseline, annot=True, fmt='d', cmap='Blues', ax=ax1,
            xticklabels=target_names, yticklabels=target_names)
ax1.set_title("Confusion Matrix: Baseline SVC (15% Labeled)", fontweight='bold')
ax1.set_xlabel("Predicted")
ax1.set_ylabel("Actual")

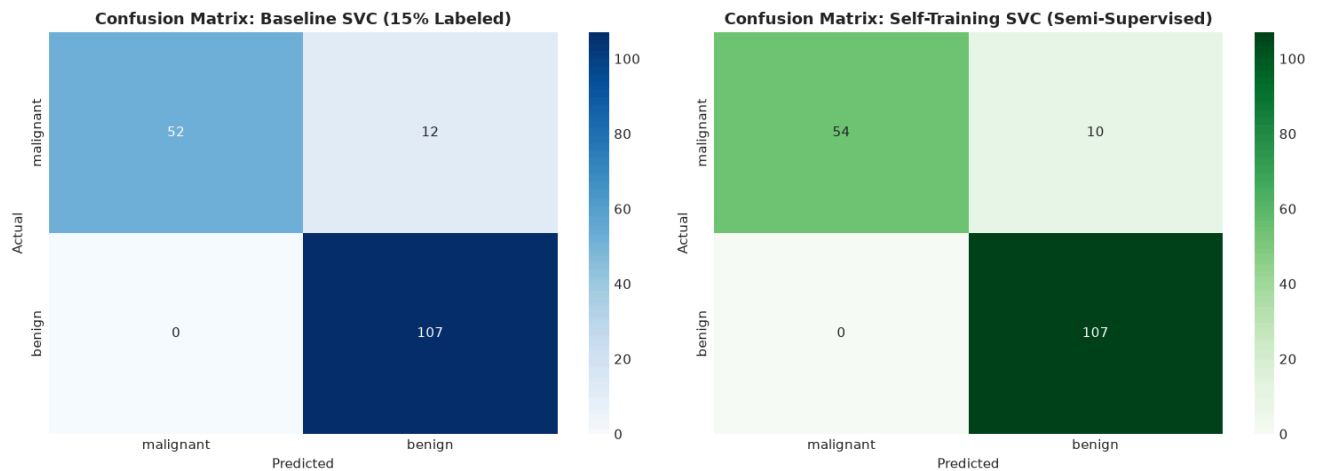
# Self-training confusion matrix
cm_semi = confusion_matrix(y_test, y_pred_semi_svc)
sns.heatmap(cm_semi, annot=True, fmt='d', cmap='Greens', ax=ax2,
```

```

xticklabels=target_names, yticklabels=target_names)
ax2.set_title("Confusion Matrix: Self-Training SVC (Semi-Supervised)", fontweight='bold')
ax2.set_xlabel("Predicted")
ax2.set_ylabel("Actual")

plt.tight_layout()
plt.show()

```



7. Conclusions & Practical Insights

1. **Unlabeled Data Value:** Self-training allows models to extract valuable structural distribution information from unlabeled data without human labeling effort.
2. **Gain Over Baseline:** On the Breast Cancer dataset, Self-Training improves test accuracy and F1 score over the 15% labeled baseline by incorporating confident pseudo-labels.
3. **Threshold Calibration:** Careful calibration of the confidence threshold τ (e.g. $\tau \in [0.80, 0.90]$) is essential to balance pseudo-label quantity with purity.